INDIAN INSTITUTE OF TECHNOLOGY MADRAS

BS Degree Programme in Data Science and Applications

BSDA2001P: Introduction to Deep Learning and Generative AI

Project Report

Smart MCQ Solver Challenge: Deep Learning and Transformer-Based Multiple Choice Question Answering

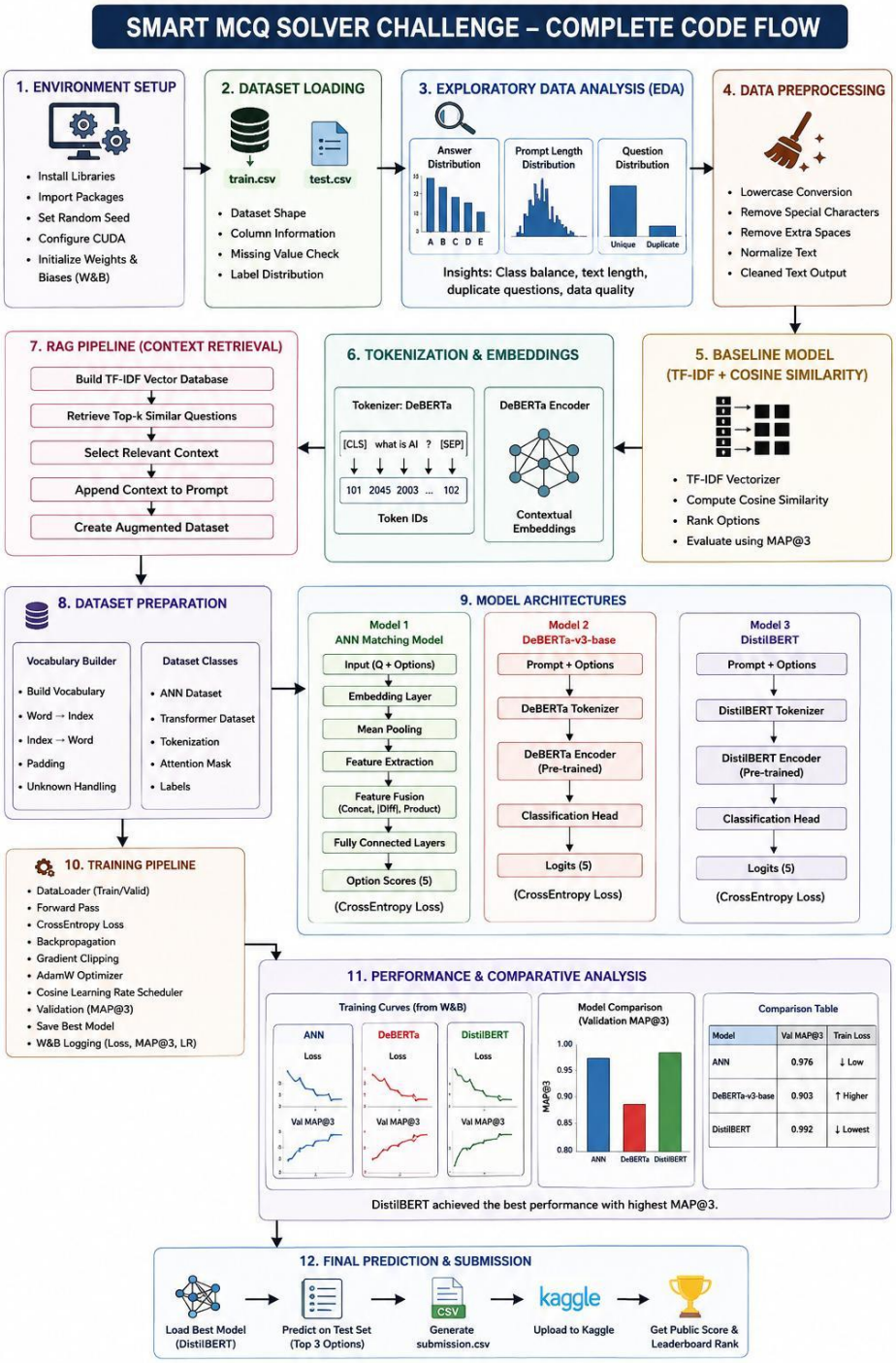
Submitted By

Name: Sagnik Sen

Roll Number: 24F1000574

Project Deployment: https://24f1000574smartmcqsolver.streamlit.app/

GitHub Repository: https: //github.com/24f1000574/dl-genai_smart-mcq-solver



Complete Code Flow – End-to-end pipeline overview

1. Abstract / Executive Summary

The Smart MCQ Solver Challenge is a Natural Language Processing (NLP) task that predicts the correct answer for multiple-choice questions using deep learning and transformer-based models. This project develops and compares approaches capable of understanding the semantic relationship between a question and its answer options.

Three models were implemented and evaluated: a custom Artificial Neural Network (ANN), DeBERTa-v3-base, and DistilBERT. Before training, the dataset was analyzed through Exploratory Data Analysis (EDA), followed by preprocessing such as text cleaning, duplicate analysis, tokenization, and retrieval-based context augmentation. The transformer models were fine-tuned using the Hugging Face Transformers library, while the ANN was built in PyTorch.

Models were evaluated using Mean Average Precision at 3 (MAP@3), the official Kaggle competition metric. DistilBERT achieved the highest validation MAP@3, followed by the ANN and DeBERTa. The project shows that transformer-based language models significantly outperform traditional bag-of-words approaches for semantic MCQ answering, while a lightweight ANN can remain competitive at much lower computational cost.

Dataset → EDA → Preprocessing → Tokenization → [ANN | DeBERTa | DistilBERT] → Evaluation (MAP@3)

2. Introduction

2.1 Problem Statement

The challenge requires selecting the correct answer from five options (A–E) for each multiple-choice question. Unlike keyword matching, this requires semantic understanding, contextual reasoning, and language comprehension. Each record contains a question prompt, five candidate answers, and the correct answer label. The objective is to train models that learn semantic relationships between questions and answer choices and maximize MAP@3.

2.2 Project Objectives

- Develop multiple deep learning models for MCQ solving.
- Compare the performance of ANN and Transformer architectures.
- Implement Hugging Face Transformers for fine-tuning.
- Perform comprehensive Exploratory Data Analysis (EDA).
- Apply modern NLP preprocessing and tokenization techniques.
- Evaluate model performance using MAP@3.
- Identify the most effective architecture for semantic MCQ answering.

2.3 Report Structure

The report covers: Dataset and Preprocessing, Exploratory Data Analysis, Tokenization Strategy, Model Development, Model Comparison, Performance Analysis, Conclusion, and References.

3. Dataset & Preprocessing

3.1 Dataset Description

The dataset was provided as part of the Smart MCQ Solver Kaggle competition and consists of two files:

- Training Dataset: 2,000 questions, 8 columns.
- Testing Dataset: 500 questions, 7 columns.

Each training sample contains a Question Prompt, Options A–E, and the Correct Answer. Each question belongs to one of five answer classes (A–E).

test.csv (461.91 kB)

Detail Compact Column

id	prompt	A	B	C	D	E
1 … 500	492 unique values	239 unique values	244 unique values	237 unique values	236 unique values	243 unique values
1	Pick the best possible answer: What is the relationship between the Hamiltonians and eigenstates in ...	For every eigenstate of one Hamiltonian, its partner Hamiltonian has a corresponding eigenstate with...	For every eigenstate of one Hamiltonian, its partner Hamiltonian has a corresponding eigenstate with...	For every eigenstate of one Hamiltonian, its partner Hamiltonian has a corresponding eigenstate with...	For every eigenstate of one Hamiltonian, its partner Hamiltonian has a corresponding eigenstate with...	For every eigenstate of one Hamiltonian, its partner Hamiltonian has a corresponding eigenstate with...
2	What is the estimated redshift of CEERS-93316, a candidate high-redshift galaxy observed by the Jame...	Approximately z = 6.0, corresponding to 1 billion years after the Big Bang.	Approximately z = 16.7, corresponding to 235.8 million years after the Big Bang.	Approximately z = 3.0, corresponding to 5 billion years after the Big Bang.	Approximately z = 10.0, corresponding to 13 billion years after the Big Bang.	Approximately z = 13.0, corresponding to 30 billion light-years away from Earth.
3	Pick the best possible answer: What is the reason for the sun appearing slightly yellowish when view...	The sun appears yellowish due to a reflection of the Earth's atmosphere.	The longer wavelengths of light, such as red and yellow, are not scattered away and are directly vis...	The sun appears yellowish due to the scattering of all colors of light, mainly blue and green, in th...	The sun emits a yellow light due to its own spectrum, which is visible when viewed from Earth.	The atmosphere absorbs the shorter wavelengths of light, such as blue and red, leaving only the long...

Figure 1: Sample of the test dataset (test.csv)

3.2 Exploratory Data Analysis (EDA)

Answer Distribution

Class B contains the largest number of samples, while class E contains the smallest. The dataset is relatively balanced overall, making it suitable for supervised learning.

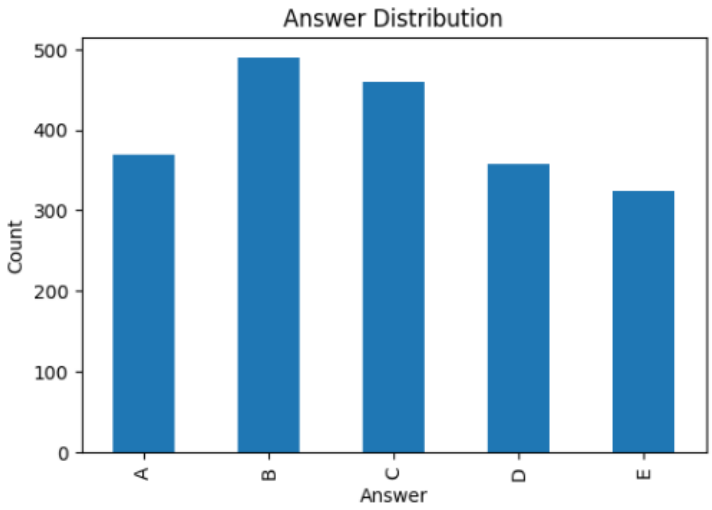


Figure 2: Answer distribution across classes A–E

Prompt Length Distribution

Most questions contain between 10 and 25 words, with very few exceeding 40 words. This justified a maximum sequence length of 128 tokens, capturing almost all question information without excessive padding.

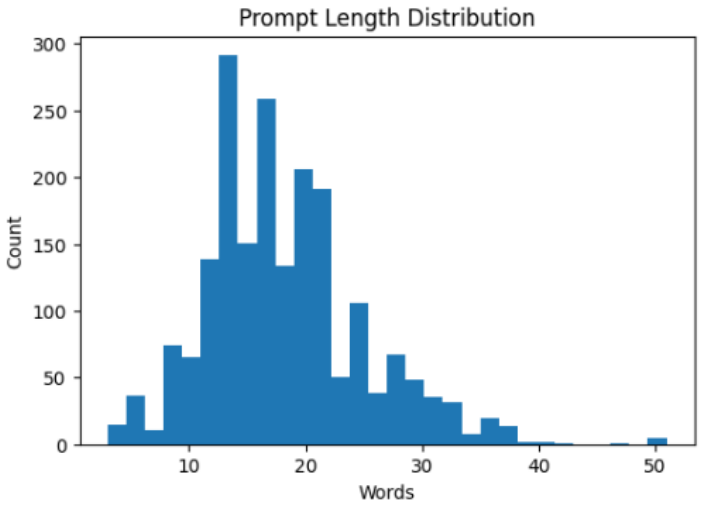


Figure 3: Prompt length distribution (in words)

Duplicate Question Analysis

The dataset contained 183 duplicate questions. These were identified and analyzed to understand redundancy in the data prior to splitting.

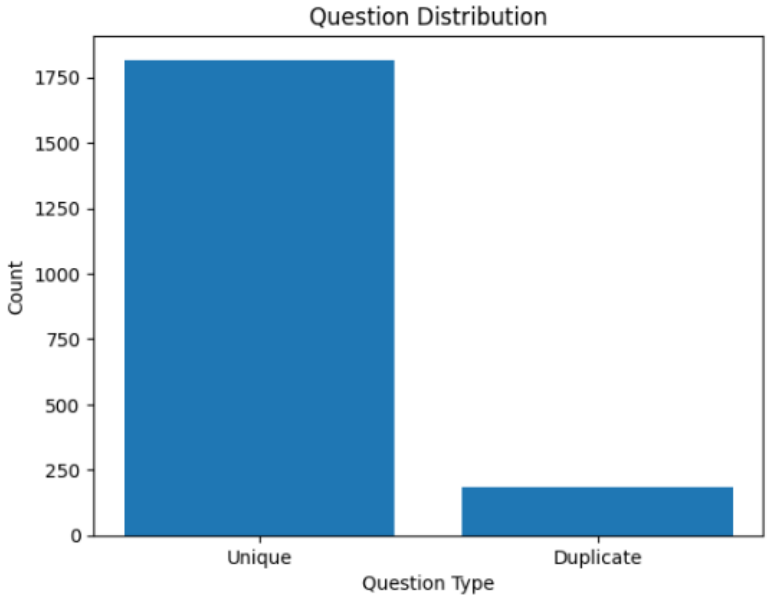


Figure 4: Unique vs. duplicate question counts

3.3 Data Preprocessing

- Converted all text to lowercase.
- Removed punctuation and special characters.
- Normalized whitespace.
- Identified and analyzed duplicate questions.
- Checked for missing values.
- Split the dataset into training and validation sets using GroupShuffleSplit.

- Applied retrieval-based context augmentation using TF-IDF similarity.
- Prepared transformer-ready input sequences.

Duplicate questions were identified and analyzed. GroupShuffleSplit was then used to split the dataset while grouping identical prompts, reducing information leakage between training and validation sets.

These techniques improved data quality and enabled efficient model training.

3.4 Data Augmentation

Instead of traditional augmentation, this project used a retrieval-based context augmentation strategy. Applied retrieval-based context augmentation using TF-IDF similarity: similar questions and their correct answers were retrieved from the training set and appended to each prompt before model training, giving the transformer models additional contextual information.

Question → TF-IDF Search → Retrieve Similar Questions → Append Context → Augmented Prompt

4. Tokenization Strategy

Tokenization converts raw text into numerical tokens for deep learning models. Since transformers cannot directly interpret plain text, each question and answer choice is transformed into token IDs while preserving semantic information. Multiple tokenization strategies were explored before finalizing the tokenizer used for training.

4.1 Primary Tokenizer Selection

The primary tokenizer was the DeBERTa-v3 Tokenizer (microsoft/deberta-v3-base), based on SentencePiece subword tokenization. For DistilBERT, the corresponding distilbert-base-uncased tokenizer was used. Both efficiently convert text into subword tokens, capturing contextual relationships even for rare or unseen words.

Each MCQ is converted into five separate input pairs by combining the prompt with each answer option (A–E), producing input IDs and attention masks. Token type IDs were included only for models that support them, since DistilBERT does not use token_type_ids while DeBERTa's tokenizer output was handled conditionally in the dataset class.

4.2 Why This Tokenization Strategy Was Chosen

- Subword tokenization reduces the number of unknown words.
- Effectively handles long sentences and complex vocabulary.
- Preserves contextual meaning between words.
- Fully compatible with pretrained DeBERTa and DistilBERT transformer models.
- Improves semantic understanding via meaningful token representations.

For DistilBERT, the official pretrained tokenizer was used to ensure compatibility with the model architecture and pretrained weights.

Raw Text → Tokenizer → Subword Tokens → Token IDs → Attention Mask → Transformer Model

4.3 Implementation Details

The Hugging Face AutoTokenizer class was used to tokenize all training and validation samples:

- Converted question-answer pairs into transformer-compatible inputs.
- Applied automatic padding (padding="max_length").
- Applied truncation for long sequences (truncation=True).
- Fixed the maximum sequence length to 128 tokens.

- Generated attention masks and, where required, token type IDs.

Batch tokenization via PyTorch DataLoader reduced preprocessing time and improved GPU utilization during training.

4.4 Experimental Comparison

TF-IDF Tokenization: Baseline using cosine similarity between question and answer options. Computationally efficient but failed to capture semantic relationships, achieving a validation MAP@3 of approximately 0.25.

DeBERTa Tokenizer: Significantly improved semantic representation via contextual embeddings, achieving a validation MAP@3 of approximately 0.903 when combined with fine-tuning.

DistilBERT Tokenizer: Achieved the best performance, with a validation MAP@3 of approximately 0.997, demonstrating excellent accuracy with fewer parameters than larger transformer models.

Transformer-based tokenization was therefore selected as the final approach due to superior contextual understanding and overall performance.

5. Modeling & Experimentation

This section presents the models implemented for the Smart MCQ Solver Challenge, comparing transformer-based architectures against a custom ANN. All models were trained and validated on the same splits and evaluated using MAP@3.

- Fine-Tuned Transformer Models (DeBERTa-v3-base and DistilBERT)
- Custom Artificial Neural Network (ANN)
- TF-IDF Semantic Similarity Baseline

5.1 Model 1: Fine-Tuned Transformer Model

Architecture

Transformer models are state-of-the-art for NLP because of their ability to model contextual relationships between words. Two pretrained models, DeBERTa-v3-base and DistilBERT-base-uncased, were fine-tuned using Hugging Face Transformers. The tokenizer combines each prompt with each answer option, producing input IDs, attention masks, and (where supported) token type IDs. These inputs pass through the transformer encoder to produce contextual embeddings, and a classification layer predicts the most probable answer among the five options.

Salient Points

- Context-aware word embeddings.
- Self-attention mechanism captures long-range dependencies.
- Excellent semantic understanding and strong transfer learning.
- Robust performance on multiple-choice question answering tasks.

Fine-Tuning Strategy

Parameter	DeBERTa	DistilBERT
Optimizer	AdamW	AdamW
Learning Rate	5 x 10^-6	2 x 10^-5
Batch Size	2	2
Epochs	10	10
Scheduler	Cosine Warmup	Cosine Warmup
Gradient Accumulation	8	8

Cross-entropy loss was used, while gradient clipping and mixed-precision training improved numerical stability and reduced GPU memory consumption. The best-performing checkpoint on validation was automatically saved via Weights & Biases (W&B).

Training Performance - DistilBERT

Validation MAP@3 increased rapidly from approximately 0.68 in the first epoch to nearly 0.997 by the final epoch, demonstrating excellent semantic learning. Training loss decreased consistently from around 3.1 to 0.07, indicating successful optimization and stable convergence. The cosine scheduler allowed large updates early and fine-grained optimization later, helping the model converge smoothly. DistilBERT achieved the highest validation MAP@3 among all evaluated models.

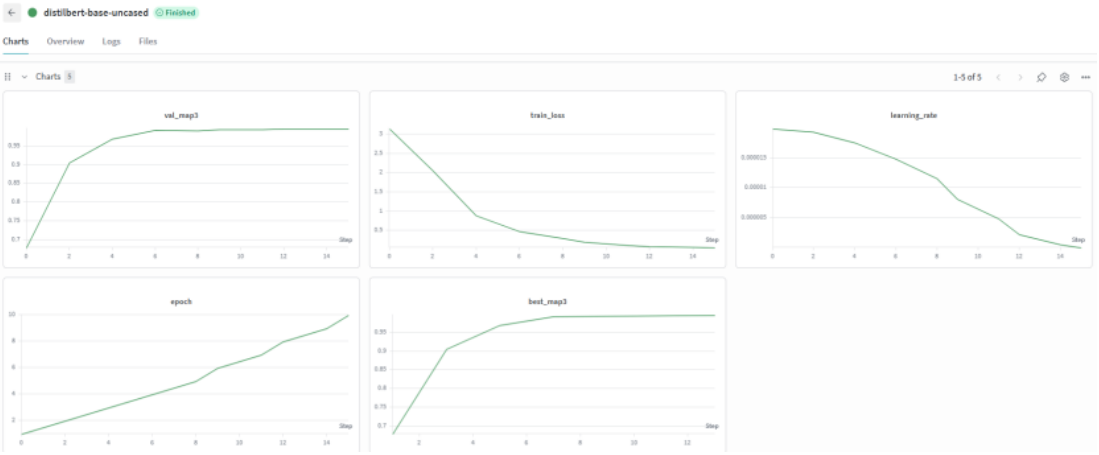


Figure 5: DistilBERT training dashboard (val_map3, train_loss, learning_rate, epoch, best_map3)

Training Performance - DeBERTa-v3-base

Validation MAP@3 improved steadily from approximately 0.54 to 0.90, while training loss decreased from nearly 3.2 to 1.34. The cosine decay schedule enabled smoother convergence in later stages. Although DeBERTa achieved lower performance than DistilBERT, it produced competitive results with strong contextual understanding.

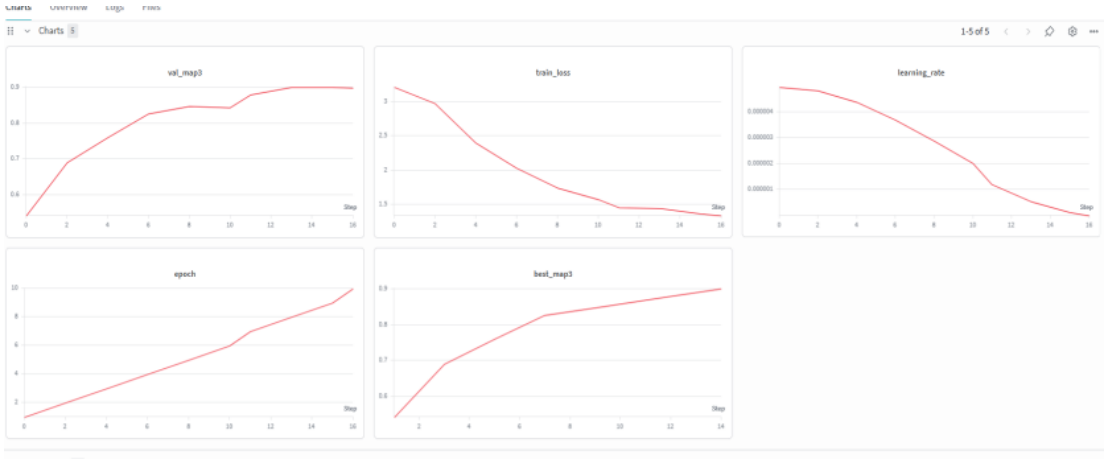


Figure 6: DeBERTa-v3-base training dashboard

5.2 Model 2: Custom Deep Learning Model (ANN)

Architecture

A custom ANN was built in PyTorch as a lightweight alternative requiring far fewer computational resources than transformer architectures. The architecture consists of an Embedding Layer, Masked Mean Pooling Layer, Feature Extraction Layer, Layer Normalization, Fully Connected Layers, GELU Activation, Dropout Regularization, and an Output Classification Layer.

The embedding layer converts token IDs into dense vectors. Masked mean pooling computes sentence embeddings while ignoring padding tokens. Feature vectors for the question and each answer option are combined using concatenation, absolute difference, and element-wise multiplication before passing through fully connected layers to predict a matching score.

Salient Points

- Lightweight architecture with faster training than transformers.
- Lower memory consumption and effective feature fusion.
- Stable optimization via Layer Normalization; reduced overfitting via Dropout.
- Efficient semantic matching between questions and answer options.

Training Performance

Validation MAP@3 increased from approximately 0.86 to 0.98 over training, while training loss decreased steadily from around 1.30 to 0.57. A cosine scheduler gradually reduced the learning rate, enabling efficient updates. The ANN converged faster than the transformer models due to its smaller size and fewer parameters, offering excellent performance at considerably lower computational cost.

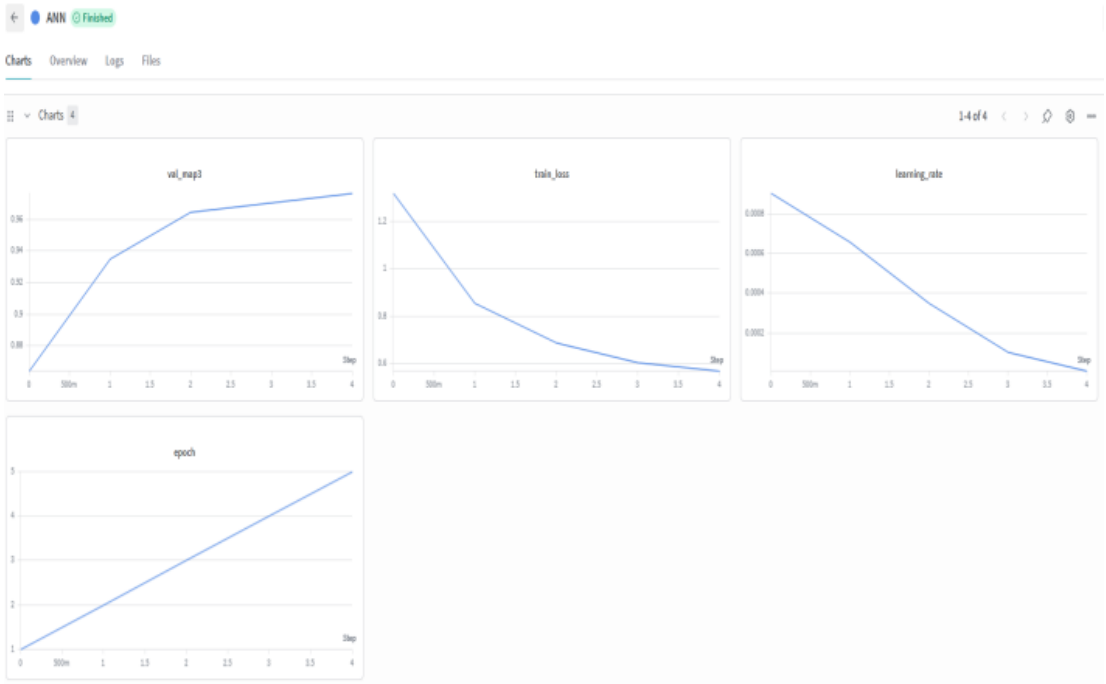


Figure 7: ANN training dashboard (val_map3, train_loss, learning_rate, epoch)

5.3 Model 3: TF-IDF Semantic Similarity Baseline

Architecture

As a benchmark, prompts and answer options were converted into TF-IDF vectors using scikit-learn's TfidfVectorizer. Cosine similarity was computed between the question and each option, and the option with the highest similarity was selected. This baseline does not require model training and relies only on lexical similarity between prompts and answer options.

Salient Points

- Fast execution and simple implementation.
- No model training required; low computational cost.
- Useful as a performance baseline.

However, it cannot capture contextual meaning or semantic relationships, resulting in lower accuracy than the deep learning models.

Experimental Results

The TF-IDF baseline achieved a validation MAP@3 of approximately 0.25, significantly lower than the ANN and transformer models, highlighting the limits of Bag-of-Words representations for semantic MCQ answering.

Model Comparison

Model	Type	Validation MAP@3	Remarks
TF-IDF	Classical NLP	0.25	Baseline model with lexical similarity
ANN	Custom Deep Learning	0.9768	Lightweight model with strong performance
DeBERTa-v3-base	Transformer	0.9029	Good contextual understanding
DistilBERT	Transformer	0.9974	Best-performing model with highest accuracy

6. Performance & Comparative Analysis

This section evaluates all implemented models using the official Kaggle metric, MAP@3, along with training loss, learning-rate progression, and computational efficiency, to identify the most suitable model.

6.1 Evaluation Metrics

MAP@3 measures whether the correct answer appears within the top three predicted options; a higher score indicates better performance. Training loss, validation MAP@3, learning-rate progression, and training stability were also monitored to assess convergence, optimization, and generalization.

6.2 Training Performance Summary

ANN: Validation MAP@3 rose from 0.8643 to 0.9768 over five epochs, with training loss falling from ~1.30 to 0.57. A cosine scheduler contributed to stable convergence at low computational cost.

DeBERTa-v3-base: Validation MAP@3 increased from 0.5430 to 0.9029 over ten epochs, with training loss decreasing from ~3.2 to 1.34, indicating stable but comparatively slower convergence.

DistilBERT: Validation MAP@3 rose rapidly from 0.6796 to 0.9974 over ten epochs, with training loss falling from ~3.1 to 0.07 — the best convergence and minimal overfitting of all models tested.

6.3 Comparative Report

Table 6.1 summarizes the performance of all models.

Model	Type	Val. MAP@3	Epochs	Remarks
TF-IDF	Classical NLP	0.2500	–	Baseline model
ANN	Custom Deep Learning	0.9768	5	Lightweight and efficient

DeBERTa-v3-base	Transformer	0.9029	10	Good contextual understanding
DistilBERT	Transformer	0.9974	10	Best overall performance

Transformer-based models significantly outperform traditional NLP approaches. DistilBERT achieved the highest MAP@3 while maintaining stable optimization; the ANN also performed exceptionally well with far fewer computational resources, making it well suited to hardware-constrained settings.

6.4 Kaggle Performance

The final submission was generated using DistilBERT, the best-performing model on validation MAP@3, and was selected for predictions on the Kaggle test dataset. Final public/private leaderboard scores and rank are to be added once available.

7. Conclusion & Future Work

7.1 Key Learnings

This project provided practical experience applying NLP and deep learning to a real-world multiple-choice question answering task, including dataset preprocessing, tokenization, transformer fine-tuning, custom neural network development, and evaluation using MAP@3. It also strengthened understanding of PyTorch, Hugging Face Transformers, Weights & Biases, and modern NLP workflows, and highlighted the advantages of contextual embeddings over traditional feature-based approaches.

7.2 Challenges Faced

- Handling transformer input formatting for multiple-choice questions.
- Managing GPU memory limitations during training.
- Selecting appropriate learning rates and batch sizes.
- Preventing overfitting during fine-tuning.
- Choosing the best-performing model among multiple architectures.

These challenges were addressed through careful hyperparameter tuning, gradient accumulation, cosine learning-rate scheduling, and continuous monitoring using W&B.

7.3 Future Work

- Train larger transformer models such as RoBERTa or DeBERTa-v3-large.
- Apply ensemble learning by combining multiple transformer models.
- Use FAISS for faster retrieval.
- Replace TF-IDF retrieval with Sentence Transformers.
- Explore Cross-Encoder retrieval for better contextual matching.
- Perform extensive hyperparameter optimization.
- Explore parameter-efficient fine-tuning techniques such as LoRA or adapters.
- Increase dataset diversity through advanced data augmentation.

These enhancements have the potential to further improve prediction accuracy and model robustness.

8. References

1. IIT Madras BS Degree Programme. BSDA2001 - Introduction to Deep Learning and Generative AI Course Page. Available at: https://study.iitm.ac.in/ds/course_pages/BSDA2001.html

2. IIT Madras BS Degree Programme. BSDA2001P - Deep Learning and Generative AI Project Guidelines and Live Session Materials (Course Resources). IIT Madras BS Degree Programme, 2026.

3. BSDA2001: Introduction to Deep Learning and Generative AI - Official IIT Madras Live Sessions (2026 Term). IIT Madras BS Degree Programme.